Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT - 10ma Edicion



Instructor: Cristian Adrian Diaz Garcia

Fecha: 22 de agosto de 2026

UNIVERSIDAD NACIONAL DE INGENIERÍA

CURSOS GRATUITOS

OTI UNI

TRANSFORMACIÓN digital

SESIÓN 6

SEGURIDAD, BACKUPS Y DESPLIEGUE FINAL

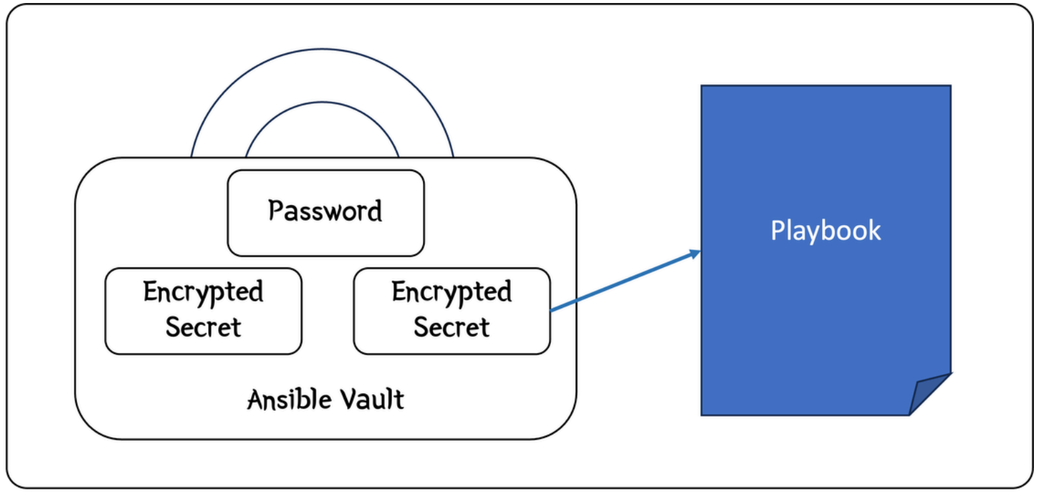
SESIÓN 6

- Protección de datos sensibles con Ansible Vault.
- Hardening SSH, gestión de usuarios, llaves, sudo y firewall UFW.
- Protección contra accesos no autorizados con Fail2Ban.
- Backups automatizados con cron, debugging y despliegue completo desde cero.

Ansible Vault

Cifrado de Secretos con Ansible Vault

- Contraseñas, tokens y claves no deben almacenarse en texto plano.
- **Ansible Vault** cifra archivos o variables sensibles mediante AES-256.
- El contenido se descifra en memoria durante la ejecución del playbook.
- El repositorio conserva código versionable sin revelar credenciales.



Regla de seguridad
El archivo de contraseña del Vault nunca se versiona ni se distribuye junto al repositorio.

Comandos esenciales

Crear un archivo cifrado desde cero

```
ansible-vault create group_vars/all/secrets.yml
```

Cifrar un archivo existente

```
ansible-vault encrypt group_vars/all/secrets.yml
```

Ver un archivo cifrado (sin modificar)

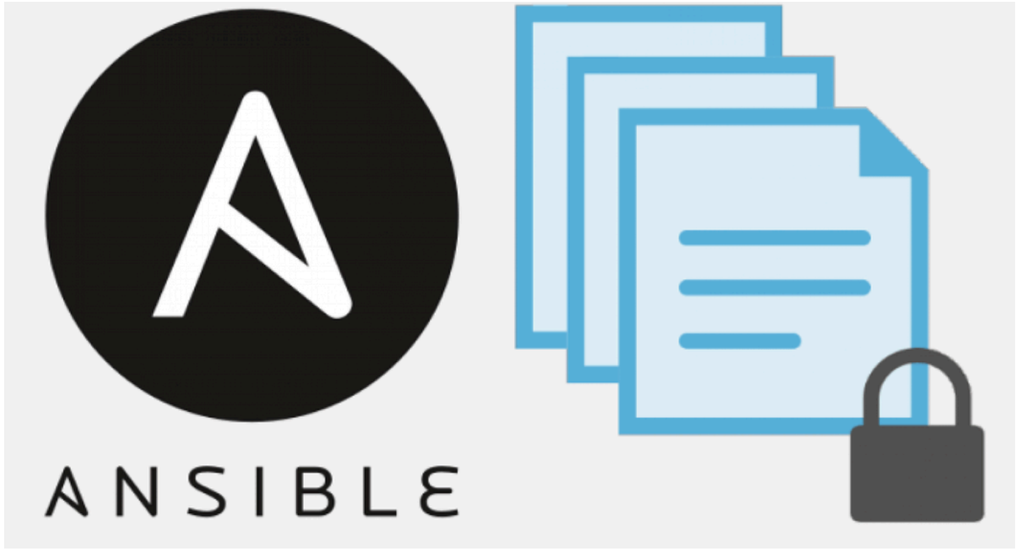
```
ansible-vault view group_vars/all/vault.yml
```

Editar un archivo cifrado

```
ansible-vault edit group_vars/all/vault.yml
```

Descifrar un archivo

```
ansible-vault decrypt group_vars/all/vault.yml
```



El patrón profesional: variables separadas (1/3)

Estructura de archivos

```
proyecto/
├── group_vars/
│   └── all/
│       ├── vars.yml       # Variables PÚBLICAS
│       └── vault.yml      # Variables SECRETAS
```



Encrypt & Decrypt Secret Data with Ansible Vault

Esta es la práctica recomendada por Red Hat y la que verás en entornos profesionales.

El patrón profesional: variables separadas(2/3)

Archivo público (group_vars/all/vars.yml)

```yaml
# Base de datos
db_host: "db.ejemplo.com"
db_port: 5432
db_name: "mi_aplicacion"
db_user: "app_user"
db_password: "{{ vault_db_password }}"  # Referencia al secreto

# API externa
api_endpoint: "https://api.ejemplo.com/v2"
api_key: "{{ vault_api_key }}"          # Referencia al secreto

# SSL
ssl_cert_path: /etc/ssl/certs/app.crt
ssl_key_path: /etc/ssl/private/app.key
ssl_key_content: "{{ vault_ssl_key }}"  # Referencia al secreto
```

El patrón profesional: variables separadas(3/3)

Archivo cifrado (vault.yml)

`ansible-vault create group_vars/all/vault.yml`

```
# group_vars/all/vault.yml (cifrado)
# Solo contiene los valores sensibles con prefijo vault_

vault_db_password: "P@ssw0rd_Pr0duct10n_2025!"
vault_api_key: "sk-abc123def456ghi789"
vault_ssl_key: |
  -----BEGIN RSA PRIVATE KEY-----
  MIIEowIBAAKCAQEA7...
  -----END RSA PRIVATE KEY-----
```

Ahora ya podrías ejecutar tu playbook normalmente, y Ansible se encargará de descifrar vault.yml en memoria para resolver las referencias en vars.yml.

Variables cifradas inline

Puedes cifrar una sola variable dentro de un archivo YAML sin cifrar el archivo completo.

```
# Cifrar un valor individual
ansible-vault encrypt_string 'password' --name 'db_password'

New Vault password:
Confirm New Vault password:
Encryption successful
db_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          32646366393335393223531333
          32646366393335393223531333
          32646366393335393223531333
          ...
```

group_vars/all/vars.yml

```
# group_vars/all/vars.yml
db_host: "db.ejemplo.com"
db_port: 5432
db_password: !vault |
 $ANSIBLE_VAULT;1.1;AES256
 32646366393335393223531333
 32646366393335393223531333
 ...
```

¿Cuándo usar inline vs archivo completo?
Cuando solo tienes 1-2 secretos en un archivo con muchas variables públicas

Vault IDs: múltiples contraseñas

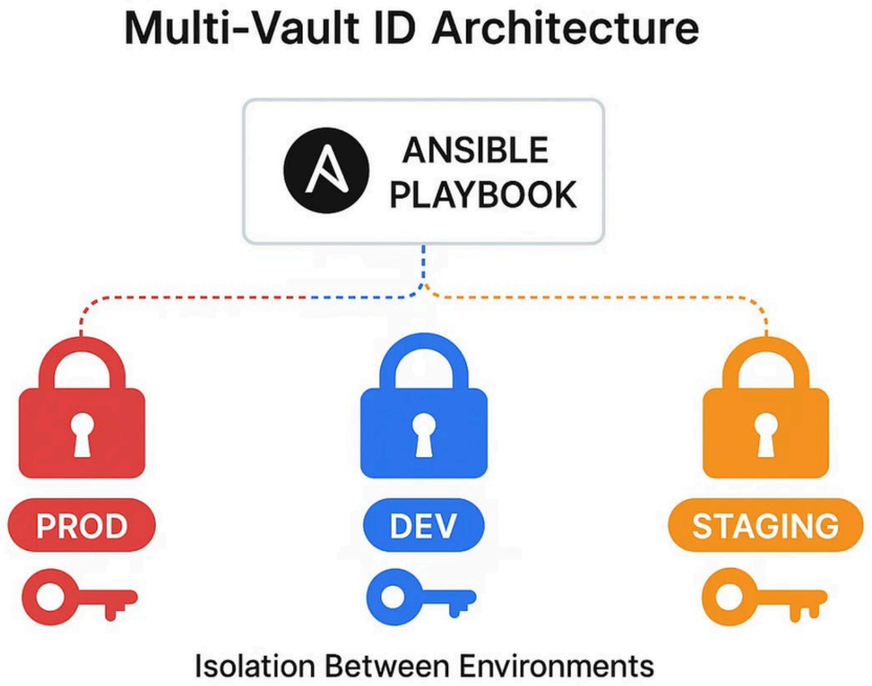
Permite etiquetar archivos o variables cifradas con un identificador de clave específico.

Cifrar con un Vault ID específico

```
ansible-vault create --vault-id dev@prompt \
  group_vars/development/vault.yml
ansible-vault create --vault-id prod@prompt \
  group_vars/production/vault.yml
```

Ejecutar playbook con múltiples Vault IDs

```
ansible-playbook site.yml \
 --vault-id dev@~/.vault_pass_dev \
 --vault-id prod@~/.vault_pass_prod
```



Archivos de contraseña por entorno (test)

Crear archivos de contraseña (uno por entorno)

```
echo "contraseña_dev_2025" > ~/.vault_pass_dev
echo "contraseña_prod_2025" > ~/.vault_pass_prod
```

Permisos estrictos (IMPORTANTE)

```
chmod 600 ~/.vault_pass_dev ~/.vault_pass_prod
```

Con esto ya no necesitas pasar --vault-id en cada ejecución. ***(NO USAR EN PRODUCCION)***

Configurar en ansible.cfg

```
[defaults]
vault_identity_list = dev@~/.vault_pass_dev, prod@~/.vault_pass_prod
```

Buenas prácticas de Vault (1/2)

Usa el prefijo *vault_* para secretos

```
# BIEN: prefijo claro
vault_db_password: "secreto"
vault_api_key: "clave"

# MAL: sin prefijo, confuso
# ¿Es la variable real o la cifrada?
db_password: "secreto"
```

Nunca descifres en producción

```
# MAL: Descifrar archivo
ansible-vault decrypt secrets.yml

# BIEN: Solo editar o ver
ansible-vault edit secrets.yml
ansible-vault view secrets.yml
```

Buenas prácticas de Vault (2/2)

Protege el archivo de contraseña

```
# Permisos restrictivos
chmod 600 ~/.vault_pass

# Añadir a .gitignore
echo ".vault_pass" >> .gitignore
echo "*.vault_pass" >> .gitignore
```

Usa *no_log* para tareas con secretos

```
- name: Configurar contraseña de base de datos
  mysql_user:
    name: app_user
    password: "{{ db_password }}"
    state: present
  no_log: yes
# Evita que la contraseña aparezca en los logs
```

Rota secretos periódicamente!!

Hardening SSH: Orden Seguro de Aplicación

1.- Crear o confirmar el usuario administrativo remoto.
2.- Instalar y probar la llave pública con *authorized_key*.
3.- Validar una segunda conexión SSH en otra terminal.
4.- Desactivar *PermitRootLogin* y *PasswordAuthentication*.
5.- Validar *sshd -t* antes de recargar el servicio.
6.- Aplicar UFW permitiendo SSH antes de habilitar la política *deny*.



Evitar el lockout
Nunca cierres la sesión actual hasta confirmar que la autenticación por llave funciona y el puerto SSH permanece permitido.

Rol de Seguridad: SSH y UFW

Configurar SSH de forma validada

```
- name: Publicar configuracion SSH
  ansible.builtin.template:
    src: sshd_config.j2
    dest: /etc/ssh/sshd_config
    owner: root
    group: root
    mode: '0600'
    validate: '/usr/sbin/sshd -t -f %s'
  notify: Recargar ssh

- name: Instalar llave autorizada
  ansible.posix.authorized_key:
    user: "{{ admin_user }}"
    key: "{{ lookup('file', admin_pubkey) }}"
```

Permitir solo servicios necesarios

```
- name: Permitir puertos autorizados
  community.general.ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  loop: "{{ firewall_allowed_ports }}"

- name: Activar UFW con politica deny
  community.general.ufw:
    state: enabled
    policy: deny
    direction: incoming
```

Valor sugerido

firewall_allowed_ports: ['22', '80', '443']

Protección frente a Intrusos con Fail2Ban

- Analiza logs de autenticación.
- Detecta intentos fallidos repetidos.
- Aplica bloqueos temporales mediante el firewall.
- Complementa, pero no reemplaza, la autenticación por llaves.

Valores iniciales
maxretry=3, findtime=10m y bantime=1h; deben ajustarse al riesgo y operación del entorno.

```
- name: Instalar Fail2Ban
  ansible.builtin.package:
    name: fail2ban
    state: present

- name: Configurar jail SSH
  ansible.builtin.template:
    src: jail.local.j2
    dest: /etc/fail2ban/jail.local
  notify: Reiniciar fail2ban
```

Backups Automatizados y Verificables

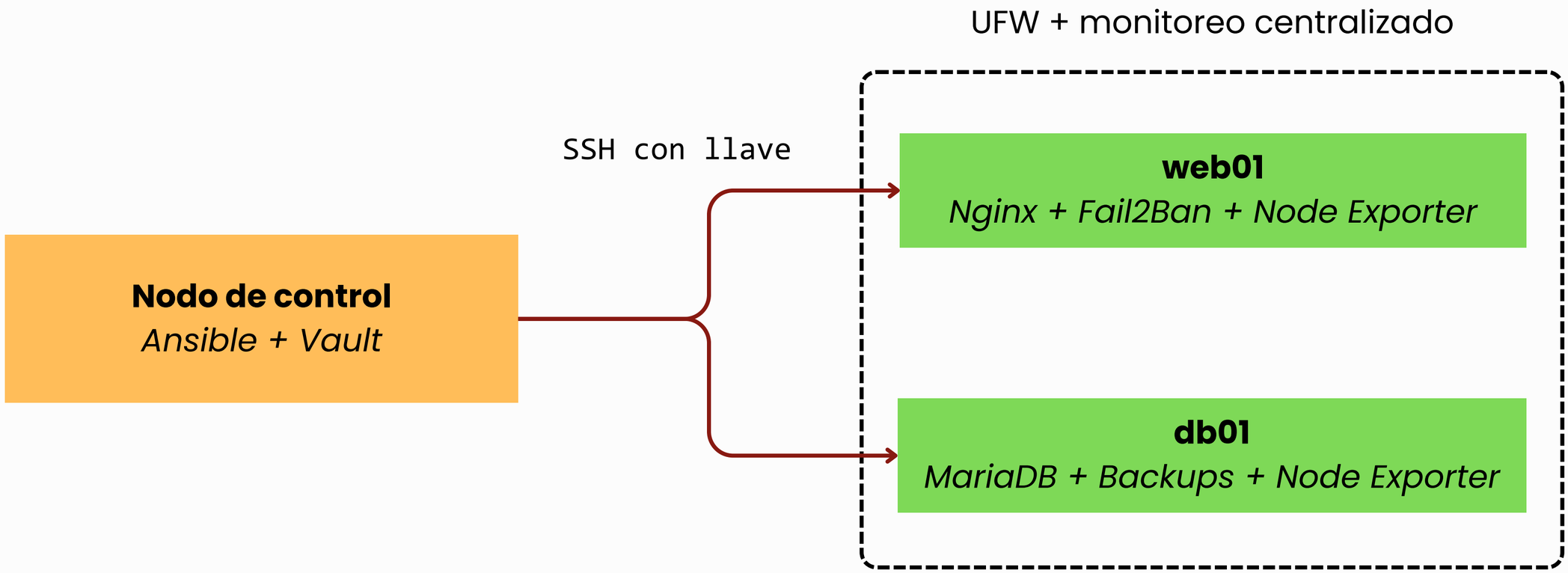
```
- name: Crear directorio de backups
  ansible.builtin.file:
    path: /var/backups/mariadb
    state: directory
    owner: root
    group: root
    mode: '0700'

- name: Instalar script de backup
  ansible.builtin.template:
    src: backup_mariadb.sh.j2
    dest: /usr/local/sbin/backup_mariadb
    mode: '0700'
```

```
- name: Programar backup diario
  ansible.builtin.cron:
    name: backup_mariadb
    minute: '0'
    hour: '2'
    user: root
    job: /usr/local/sbin/backup_mariadb

- name: Aplicar retencion de 7 dias
  ansible.builtin.cron:
    name: limpiar_backups
    minute: '30'
    hour: '2'
    job: "find /var/backups/mariadb \
      -type f -mtime +7 -delete"
```

Arquitectura del Proyecto Final



Laboratorio Final: Alcance y Entregables

Objetivo
Desplegar una infraestructura segura y observable mediante un único comando, reutilizando los roles construidos durante el curso.

***Ventajas*:**

- Web: Nginx y contenido dinámico.
- DB: MariaDB y backup programado.
- Todos: hardening, UFW y Node Exporter.

Entregables

- Inventario y configuración de Ansible.
- Roles documentados y parametrizados.
- Secretos cifrados con Vault.
- *site.yml* y *requirements.yml*.
- Evidencias de validación e idempotencia.

Laboratorio Final: Estructura del Repositorio

```
ansible-final/
|-- ansible.cfg
|-- inventory.ini
|-- requirements.yml
|-- site.yml
|-- group_vars/
| |-- all/main.yml
| `-- all/vault.yml #cifrado
`-- roles/
|-- common/
|-- security/
|-- webserver/
|-- database/
|-- node_exporter/
`-- backup/
```

Control de versiones

Versionar código, templates y archivos Vault cifrados. Excluir claves privadas, .vault-pass, reintentos y artefactos temporales.

Laboratorio Final: Inventario y Variables

inventory.ini

```
[web]
web01 ansible_host=10.10.10.21

[db]
db01 ansible_host=10.10.10.31

[linux:children]
web
db

[linux:vars]
ansible_user=ansible
ansible_python_interpreter=/usr/bin/python3
```

group_vars/all/main.yml

```
admin_user: ansible
firewall_allowed_ports: ['22']
node_exporter_port: 9100
backup_retention_days: 7
vault_db_password: "{{
vault_mariadb_password }}"
```

group_vars/all/main.yml

```
vault_mariadb_password: "CAMBIAR_Y_CIFRAR"
```

```
ansible-vault encrypt \ group_vars/all/vault.yml
```

Laboratorio Final: Orquestación con *site.yml*

```yaml
- name: Base comun y
seguridad
  hosts: linux
  become: true
  roles:
    - common
    - security
    - node_exporter

- name: Capa web
  hosts: web
  become: true
  roles:
    - webserver
```

```yaml
- name: Capa de datos
  hosts: db
  become: true
  roles:
    - database
    - backup
```

Orden de despliegue

Primero acceso y seguridad comunes; después servicios por grupo. Los handlers se ejecutan únicamente ante cambios.

Laboratorio Final: Pipeline de Ejecución

Instalar dependencias y validar inventario.

```
ansible-galaxy install -r requirements.yml
ansible-inventory -i inventory.ini --graph
ansible linux -i inventory.ini -m ping
```

Validar sintaxis y simular cambios.

```
ansible-playbook -i inventory.ini site.yml --syntax-check
ansible-playbook -i inventory.ini site.yml --check --diff --ask-vault-pass
```

Desplegar y guardar evidencia.

```
ansible-playbook -i inventory.ini site.yml --ask-vault-pass | tee despliegue.log
```

Laboratorio Final:Matriz de Validación

Componente	Prueba	Resultado esperado
SSH	Nueva conexión con llave	Acceso exitoso; root y password deshabilitados
UFW	`ufw status`	Solo puertos autorizados
Nginx	Petición HTTP/HTTPS	Código 200 y contenido correcto
MariaDB	Estado y consulta simple	Servicio activo; autenticación con Vault
Fail2Ban	`fail2ban-client status sshd`	Jail habilitada y operativa
Node Exporter	`/metrics en puerto 9100`	Métricas disponibles desde Prometheus

Laboratorio Final: Cambio Controlado e Idempotencia

Primera ejecución

Registrar *ok, changed, failed* y unreachable. Corregir cualquier fallo antes de continuar.

Segunda ejecución

```
ansible-playbook -i inventory.ini \
site.yml --ask-vault-pass
```

Resultado esperado: *changed=0.*

Cambio controlado

Modificar el contenido web o un parámetro permitido. Debe cambiar únicamente el host y rol afectados; el handler correspondiente se ejecuta una sola vez.

Evidencia
Comparar los tres recap: despliegue inicial, segunda corrida idempotente y cambio controlado.

Errores Frecuentes del Laboratorio Final

Síntoma	Causa probable	Revisión
UNREACHABLE	IP, usuario o llave incorrectos	Inventario, SSH y permisos de la llave
Lockout SSH	UFW o hardening aplicado antes de validar	Consola, llave autorizada y puerto 22
Vault no descifra	Clave o archivo equivocado	Vault ID y origen de la contraseña
Handler no corre	Tarea sin cambio o nombre distinto	notify, listen y resultado
Target DOWN	Servicio o firewall	systemd, puerto 9100 y Prometheus
Backup vacío	Credenciales, ruta o permisos	Script manual, logs y restauración

Gracias!!!

